Project 3 - Building a Vision Transformer

Om Sanan

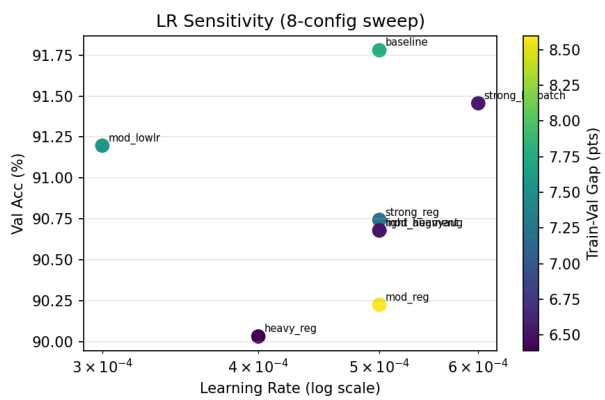
Part 1: Training Basics

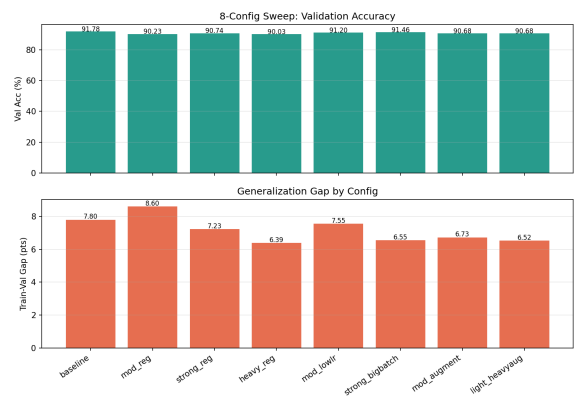
- How did you train your model? What loss function did you use, and why?

MNIST pretrain (20 epochs).
MNIST-in-the-wild finetune (150 max epochs, early stopping).
Used CrossEntropyLoss(label_smoothing=0.1) because it's standard for multiclass classification and does label smoothing.

- What learning rate and batch size did you choose? How sensitive was training to these choices? Show any figures you have.

Finetune lr was 6e-4, batch size 128, optimizer AdamW (train_final.py:25).
I ran hyperparam tuning varying lr, dropout, drop_path, weight_decay, label_smooth, and augmentation strength (results in tuning_results.json). LR in the 5e-4 to 6e-4 range worked best. Too weak regularization increased the train/val accuracy gap, and too strong regularization lowered val accuracy.





- How long did you train for? Why did you choose that number of epochs? Show any figures you have.

Pretrain: 20 epochs.
Finetune: 150 max epochs with early stopping (patience 25)
20 pretrain epochs was enough to learn solid digit primitives from clean MNIST
Longer finetune window needed for adaptation to the noisy/varied in-the-wild images; early stopping kept it from overfitting.

- How deep is your model, and how many attention heads did you use?

4 transformer layers, 8 attention heads, 256 embedding dimension

- How does your model perform during validation? Why do you believe your validation strategy is reliable?

Latest eval of my best checkpoint: train(no aug) 96.93%, val 96.25% (on average a little less accuracy)
Validation used a fixed random seed split (85/15), no val augmentation, and direct train (no aug).

- Why did you make these architectural and optimization choices overall?

Hybrid CNN+Transformer was chosen to combine CNN local bias (given limited data) with transformer global context reasoning/attention. The optimization/regularization (AdamW, weight decay, label smoothing, dropout, drop path, augmentation) was chose to control overfit while preserving accuracy across varied in-the-wild conditions.

Part 2: Data Loading and Preprocessing

- How does your dataloader work? How did you split your training and validation sets?

Data loaded from data/dataset JPG files Split: 85% train / 15% val.

- What preprocessing or data augmentation steps are applied to the images?

Train: Resize(160,160), RandomResizedCrop(128, scale=(0.5,1.0), ratio=(0.75,1.33)), RandomRotation(20), RandomPerspective(p=0.3, distortion_scale=0.3), Gaussian blur, ColorJitter(brightness=0.4, contrast=0.4), Normalize([0.5]*3,[0.5]*3), RandomErasing(p=0.3). Val: Resize(128,128), Normalize.

- Why are these processing steps appropriate for this task?

The task involves viewpoint changes, blur, occlusion, scale variation, and lighting variability. The augmentation stack explicitly simulates those effects (random crop for scale, rotation for angle, perspective for viewpoint, color jitter for lighting, erasing for occlusion) while preserving the specific digits to account for variance in how classmates took the images. Basically same reasoning as last project.

- Which invariances (if any) are you encouraging through your data pipeline?

Scale invariance (RandomResizedCrop), small rotation invariance (RandomRotation), mild perspective invariance (RandomPerspective), brightness/contrast invariance (ColorJitter for brightness/contrast), and partial occlusion (RandomErasing).

Part 3: Model Inspection and Interpretation

- Include the output of print(model) in your writeup. For each line or block in the printed output, explain what operation it performs.

```
HybridViT(
 (tokenizer): Sequential(                                                    CNN
  (0): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)        extract local
  (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)     edges/strokes
  (2): GELU(approximate='none')                                                            and
  (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)          downsample.
  (4): BasicBlock(                                                           Refine local features (residual add)
   (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
   (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
   (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
   (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
   (act): GELU(approximate='none')
   (shortcut): Identity()
  )
  (5): BasicBlock(                                                           grow channels, reduce resolution
   (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)   add projection shortcut
   (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
   (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
   (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

```
    (act): GELU(approximate='none')
    (shortcut): Sequential(
      (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (6): BasicBlock(                                                    deeper semantic features for the whole digit
    (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (act): GELU(approximate='none')
    (shortcut): Sequential(
      (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
)
(proj): Identity()                                                    Identity: no-op, placeholder
(pos_drop): Dropout(p=0.2, inplace=False)                             Regularization before transformer
(transformer_blocks): ModuleList(
  (0): TransformerBlock(
    (norm1): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
    (attn): MultiheadAttention(
      (out_proj): NonDynamicallyQuantizableLinear(in_features=256, out_features=256, bias=True)
    )
    (dp1): Identity()
    (norm2): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
    (mlp): Sequential(
      (0): Linear(in_features=256, out_features=1024, bias=True)
      (1): GELU(approximate='none')
      (2): Dropout(p=0.2, inplace=False)
      (3): Linear(in_features=1024, out_features=256, bias=True)
      (4): Dropout(p=0.2, inplace=False)
    )
    (dp2): Identity()
  )
  (1-3): 3 x TransformerBlock(                  Norm -> MultiheadAttention -> residual -> Norm -> MLP(256->1024->256) -> residual.
    (norm1): LayerNorm((256,), eps=1e-05, elementwise_affine=True)        Block 0 uses identity drop-path
    (attn): MultiheadAttention(                                           Blocks 1-3 use DropPath for depth regularization.
      (out_proj): NonDynamicallyQuantizableLinear(in_features=256, out_features=256, bias=True)
    )                                                         each spatial token attends to all other tokens and mixes global information
    (dp1): DropPath()
    (norm2): LayerNorm((256,), eps=1e-05, elementwise_affine=True)                Final normalization
    (mlp): Sequential(
      (0): Linear(in_features=256, out_features=1024, bias=True)
      (1): GELU(approximate='none')
      (2): Dropout(p=0.2, inplace=False)
      (3): Linear(in_features=1024, out_features=256, bias=True)
      (4): Dropout(p=0.2, inplace=False)
    )
    (dp2): DropPath()
  )
)
```

```
  (norm): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
  (fc): Linear(in_features=256, out_features=10, bias=True)        Classifier logits for digits 0-9
)
```

- Identify which parts of the model are responsible for classification.

The final fc layer (Linear 256->10) is the primary classifier. The classification path goes: tokenization -> transformer blocks -> CLS token extraction -> LayerNorm -> fc logits.

- Identify which layers capture low-level visual features and which capture higher-level structure, and explain why.

Low level: early tokenizer conv and first BasicBlock learn edges, strokes, and textures.
Mid level: deeper tokenizer blocks capture digit specific structures like loops and line junctions.
High level: transformer blocks take into accoint global shape and context across all spatial tokens, which is where full digits are identified/classified.

Part 4: Model Capacity and Parameters

- How many total parameters does your model have?

4,411,722

- Which components contribute most to the parameter count?

Transformer blocks: 3,159,040 (71.6%)
Tokenizer + proj: 1,232,704 (27.9%)

- Could the model reasonably have been smaller? Larger?

Smaller: yes, and it would be faster/cheaper, but likely lower accuracy on harder wild cases
Larger: yes, but higher overfit risk and honestly diminishing returns give the dataset size

- What tradeoffs motivated your final model size?

256 dim, 4 layers, 8 heads gave a good balance: enough capacity for strong accuracy, still trainable quickly on one GPU, and manageable overfitting with the regularization

AI Usage

AI was used to code up some parts of the notebook. Specifically I used it to help choose good data transforms/augmentations before training and brainstormed with it about the architecture of the hybrid CNN/ViT as I haven't put such an architecture together before. I used OpenAI Codex CLI so I can't really export chat logs.

Notebook Link

https://colab.research.google.com/drive/16SYmHeCE-2xbxfEFJcAvG-EsbzTegkNc?usp=sharing